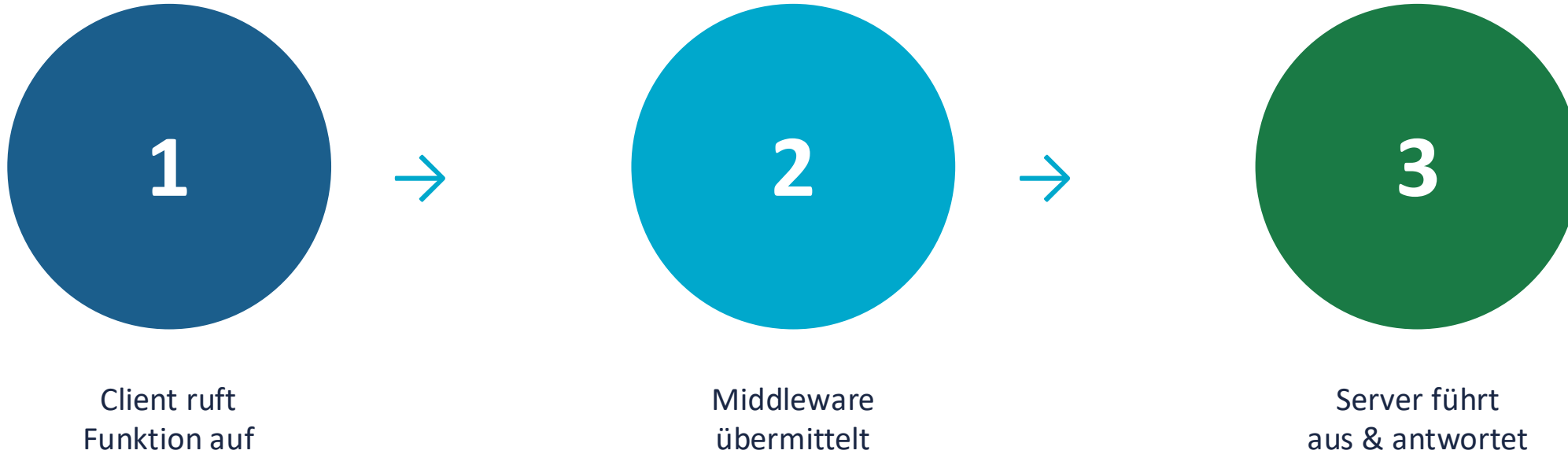


Tut09

Anne

Remote Procedure Call (RPC)

Grundidee



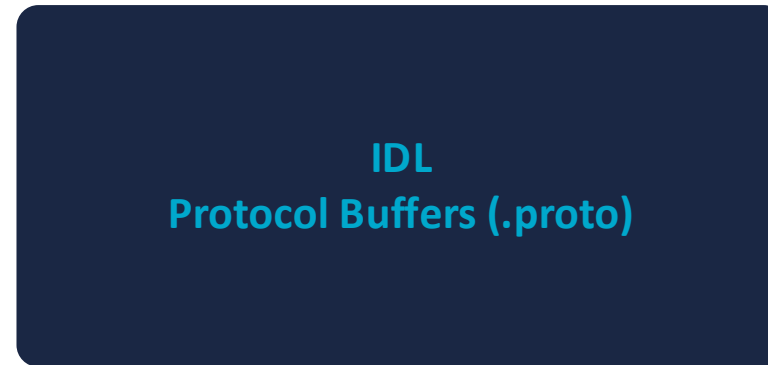
Vorteile

Remote-Aufrufe wie lokale Funktionen
Plattformunabhängig -> Einfach nutzbar

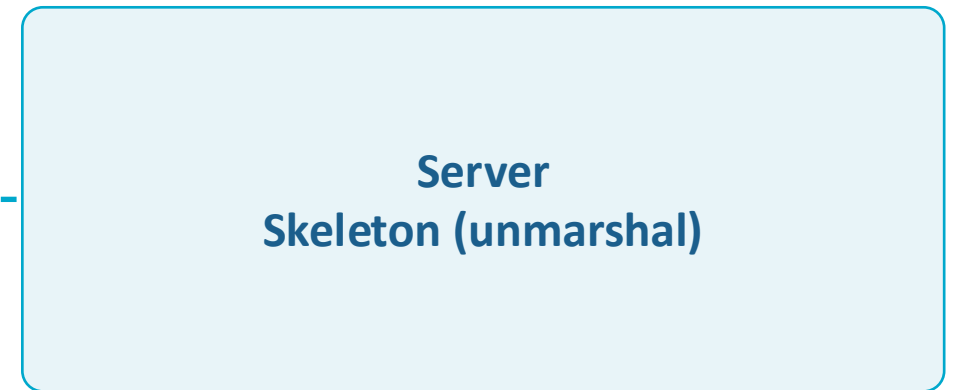
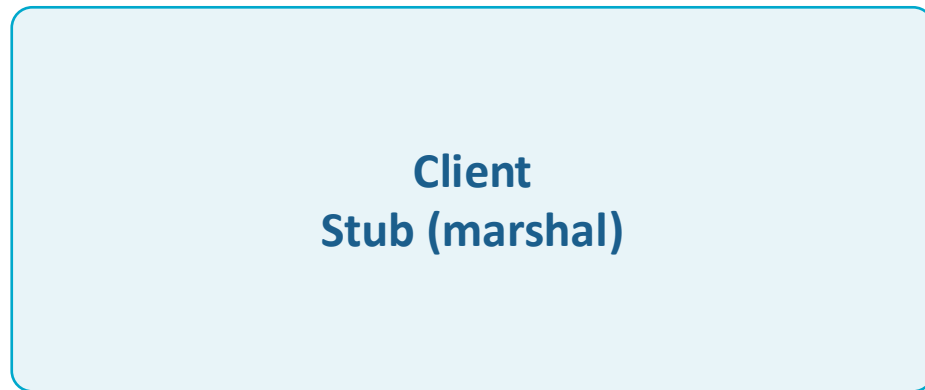
Nachteile

Synchron → Caller wird blockiert
Latenz wirkt wie lokaler Aufruf (irreführend)

IDL, Stubs & Skeletons



↓ IDL Compiler ↓



Network Binding

gRPC Workflow: 1. .proto schreiben → 2. Compiler generiert Stubs → 3. Server implementiert → 4. Client nutzt Stub

Grpc

Skeleton

HelloServiceGrpc.HelloServiceImplBase

Stub

HelloServiceGrpc.HelloServiceBlockingStub

- IDL: Protocol buffers (proto3)

```
service HelloService {  
  rpc SayHello (HelloRequest) returns (HelloResponse);  
}  
  
message HelloRequest {  
  string greeting = 1;  
}  
  
message HelloResponse {  
  string reply = 1;  
}
```

HelloRequest.java,
HelloResponse.java

Message-Klassen

Was wir schreiben müssen:

- Auf **Serverseite** generierten Skeleton extenden:

```
public class MyServer extends HelloServiceGrpc.HelloServiceImplBase {  
  
    @Override  
    public void sayHello(HelloRequest req, StreamObserver<HelloResponse> resp) {  
        // hier Logik  
    }  
}
```

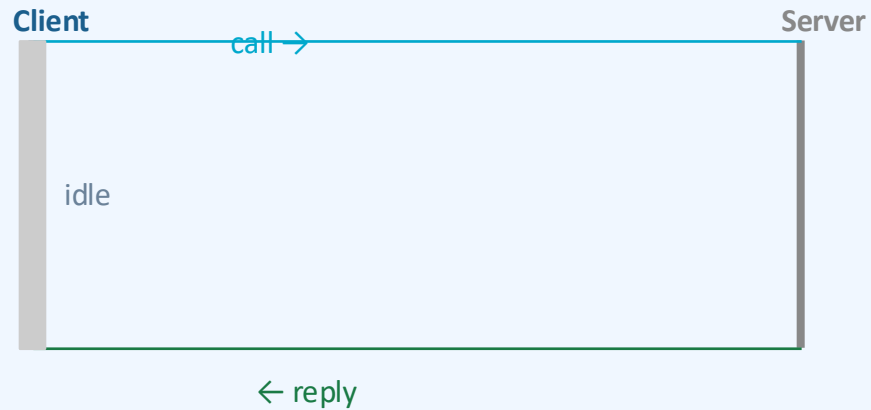
- Grpc unterstützt Streaming -> Streamobserver -> Objekt auf das wir Antworten drauf schreiben
- `resp.onNext(response)`: schick eine Antwort
- `resp.onCompleted()`: Ich bin Fertig
- `resp.onError(e)`: Fehler aufgetreten

JETZT:

Implementieren Sie einen lokal laufenden GRPC-Server und GRPC-Clients. Der Client soll einen beliebigen Namen (String) an den Server senden können, und der Server antwortet mit einer Begrüßung, die den Namen enthält

Synchron vs. Asynchron

Synchron (RPC)



- Einfach lesbar, minimale Latenz
- Sofortige Antwortverarbeitung
- Caller blockiert (idle)
- Beide müssen gleichzeitig online sein
- Kein Parallelisieren

Asynchron (Queue)



- Sender nicht blockiert
- Zeitliche Entkopplung
- Höherer Durchsatz (Throughput)
- Graceful Degradation
- Höhere Latenz möglich

Aufgabe 4: Queues

Wofür können Queues in komplexen System eingesetzt werden? Erklären Sie die Unterschiede, Nachrichten synchron oder asynchron in die Queue einzuliefern und abzuholen.

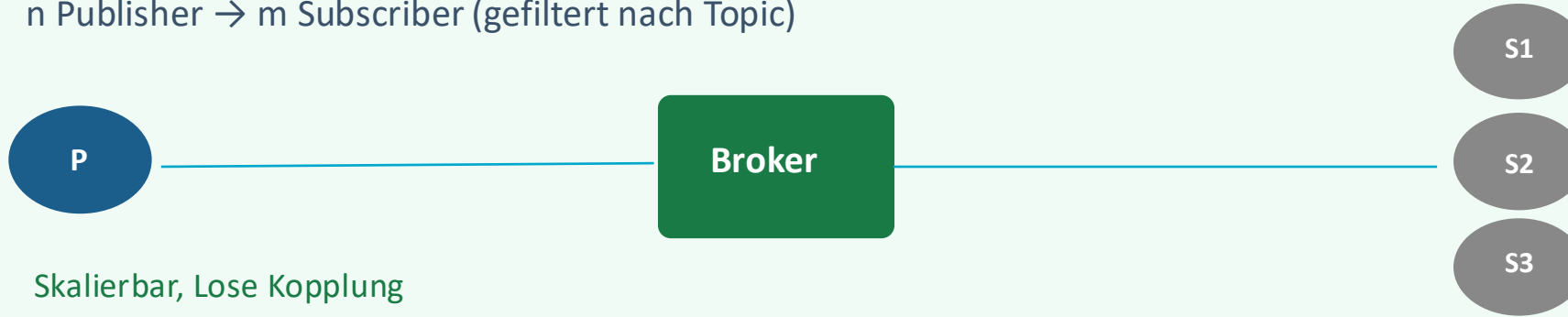
Message Queues: Senden & Empfangen

	Synchron zur Queue	Asynchron zur Queue
Sender (put)	Sicher: Queue hat Nachricht Blocking → blockiert den Sender	Minimale Latenz, Fire & Forget Sender weiß nicht ob angekommen
Empfänger (fetch)	Empfänger kontrolliert Rate (kein Overload) -> Müssen fragen Aktives Polling kostet Ressourcen	Nicht blockiert (Callback) -> Sobald Queue was hat gibt sie es uns Empfänger kann überlastet werden

Pub/Sub & Topic-basiert

Pub/Sub

n Publisher → m Subscriber (gefiltert nach Topic)



Topic-basiert :

Publisher veröffentlicht Nachricht Y zum Topic X

Subscriber abonniert Topic X → erhält alle Nachrichten dazu

Broker übernimmt Matching & Filterung

Coupling = Maß für Abhängigkeiten zwischen Komponenten -> immer minimieren!

Messaging reduziert Kopplung:

- Location-unabhängig
- keine Temporal-Abhängigkeit
- Keine Technology Abhängigkeit

Aufgabe

Erläutern Sie, wie auf Basis eines 2-Phase-Commit Protokolls entschieden werden kann, ob eine Transaktion in einer verteilten Datenbank committed oder abgebrochen wird.

2-Phase-Commit

2-Phase-Commit (2PC)

1

Phase 1: Propose

Coordinator fragt alle DB-Server: "Kannst du committen?"

2

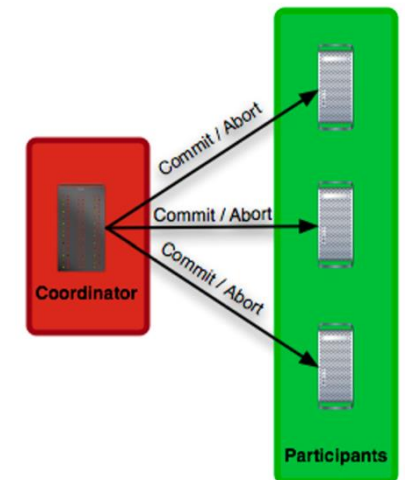
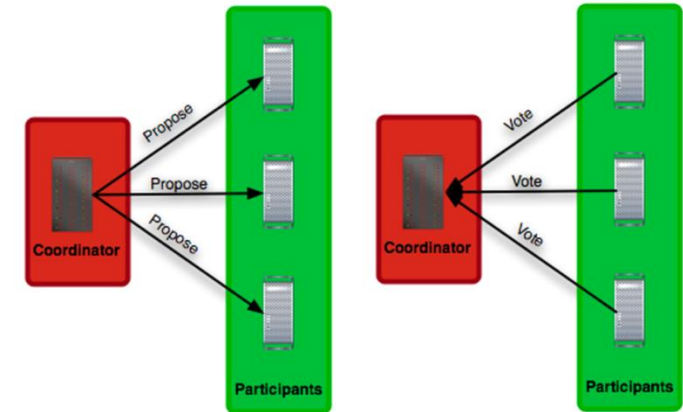
Phase 2a: Commit

Alle sagen Yes → Coordinator schickt Commit an alle

3

Phase 2b: Abort

Einer sagt No → Coordinator schickt Abort an alle



Aufgabe: Anwendungsszenarien: Welche Kommunikation?

1

Events → Push Notifications an User

Entkopplung der Produzenten von den Benachrichtigungssendern; beide können unabhängig laufen.

Queue (PTP)

2

Banken tauschen Finanztransaktionen aus

Beide Banken müssen über Kontostand einig sein. Direkte, sofortige Kommunikation erforderlich.

RPC (synchron)

3

Aktienkurse an viele Clients

Jeder Client abonniert nur die relevanten Kurse. Publisher kennen die Subscriber nicht.

Pub/Sub

4

Instant Messaging App

Online: synchron. Offline: Queue speichert Nachrichten bis User wieder online.

kombi von synchron und asynchron

Zusammenfassung

Kernaussagen

RPC:	synchron, einfach, für direkte Aufrufe
-------------	--

Messaging:	asynchron, entkoppelt, reliable
-------------------	---------------------------------

Pub/Sub:	n:m, topic-basiert, skalierbar
-----------------	--------------------------------

Loose Coupling:	immer anstreben!
------------------------	------------------

2PC:	Koordination verteilter Transaktionen
-------------	---------------------------------------

Merke: Messaging liefert Loose Coupling & RPC erzeugt tight coupling. Die Wahl hängt vom Use Case ab.

API

Eine Web API ist ein Endpunkt im Internet, über den Programme miteinander kommunizieren (meist JSON oder XML)



Wir schicken eine HTTP-Anfrage an eine URL → wir kriegen JSON zurück → wir parsen & nutzen die Daten.

HTTP-Request in Java (HttpClient)

Es gibt `java.net.http.HttpClient`

```
//1. Client erstellen (einmalig, wiederverwendbar)
HttpClient client = HttpClient.newHttpClient();

// 2. Request bauen
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.open-meteo.com/v1/forecast?latitude=52.5"))
    .GET()
    .build();

//3.Request abschicken & Antwort lesen
HttpResponse<String> response = client.send(
    request, HttpResponse.BodyHandlers.ofString());

// 4. Statuscode & Body auslesen

String body = response.body();                //JSON als String
```


Für Zuhause und die Hausaufgabe

[SoSe 2026] AS / Übungsbetrieb / Übungsaufgabe 2

Übungsaufgabe 2

Aufgabe

Einstellungen

Einreichungen

Erweiterte Bewertung

Mehr ▾

Geöffnet: Mittwoch, 3. Juni 2026, 00:00

Fällig: Freitag, 3. Juli 2026, 18:00



hello-world.zip

8. Juni 2026, 12:51



ue2-communication.pdf

3. Juni 2026, 11:53

Bewerten

Wir schreiben: `hello.proto`



protoc-Compiler: `HelloServiceGrpc.java` ← Stub + Skeleton drin
 `HelloRequest.java`
 `HelloResponse.java`



Wir schreiben: `MyServer` extends `HelloServiceImplBase` (Serverlogik)
 `MyClient` benutzt `HelloServiceBlockingStub`



Laufzeit:

Client-Code ruft `stub.sayHello(req)` auf



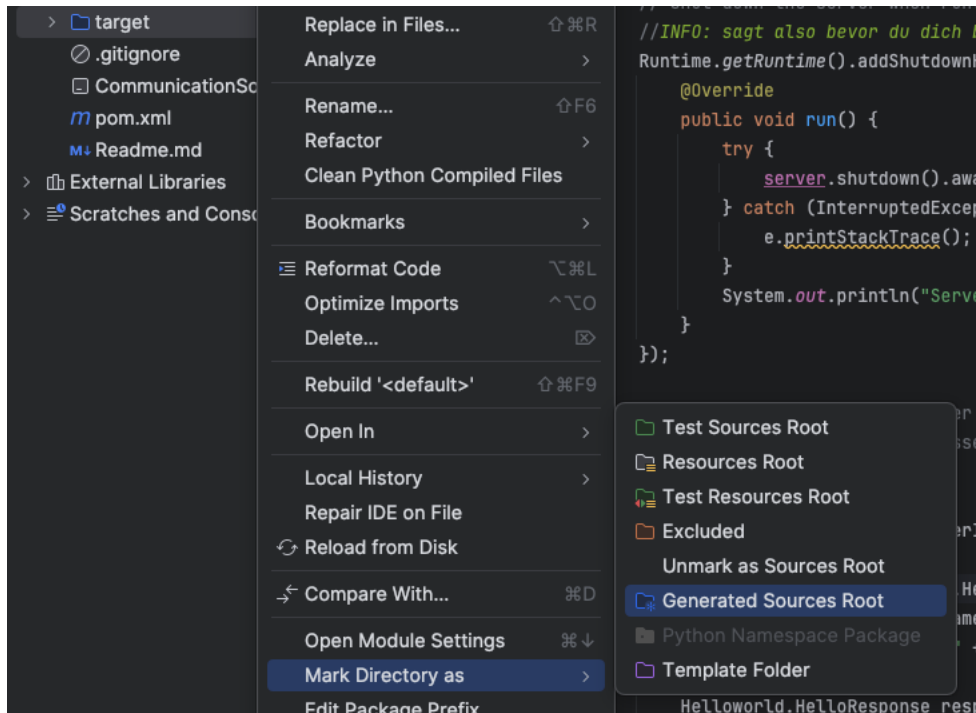
(Stub serialisiert, schickt über HTTP/2)

Skeleton empfängt, deserialisiert

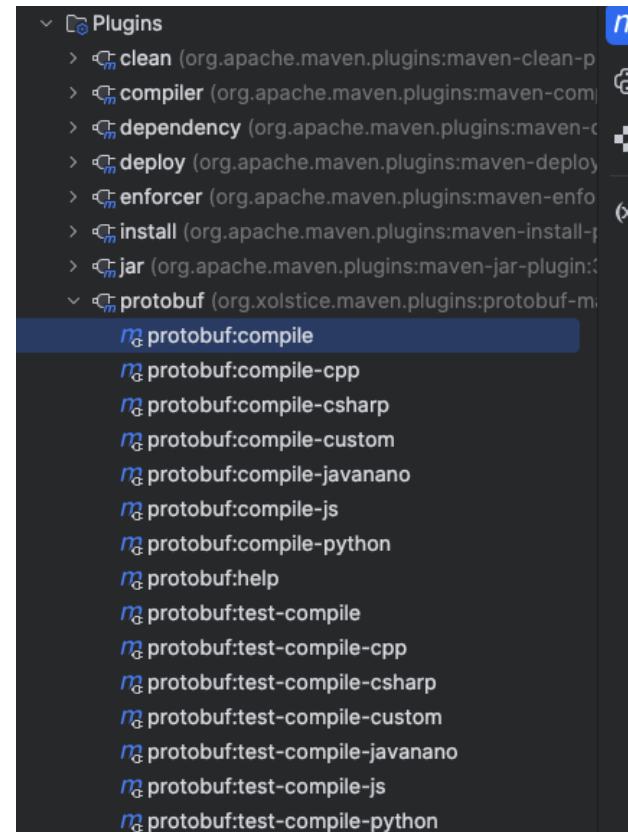


`MyServer.sayHello()` wird aufgerufen

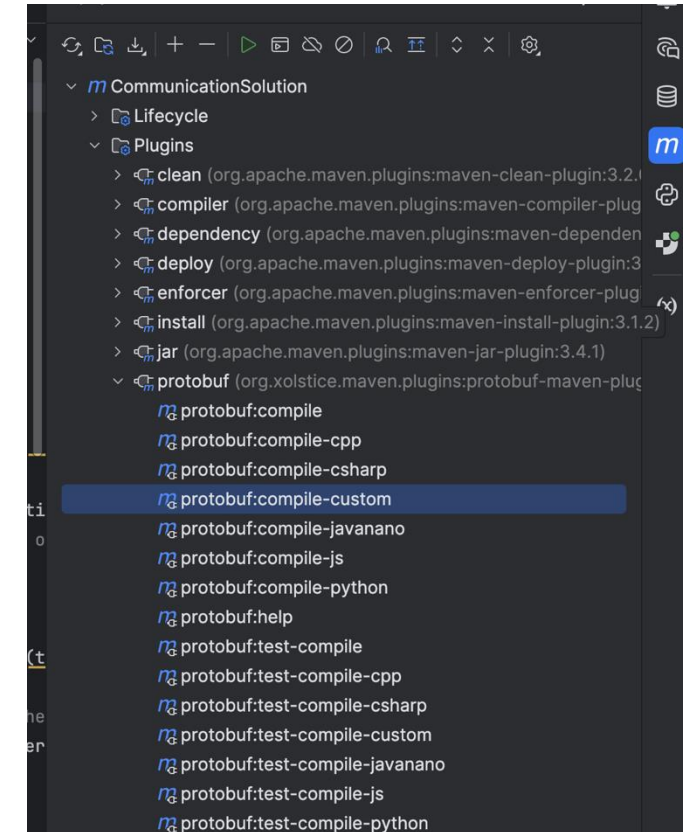
Deine Klassen finden deine .proto nicht ?



Target schon als
generated Sources Root
markiert ?



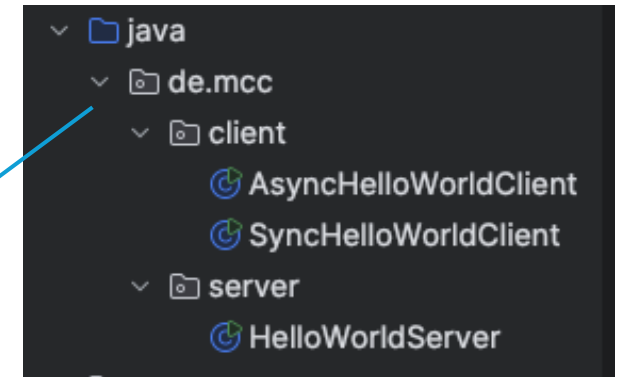
Bei m (rechts) ->
communicationSolution -> Plugins -
> protobuf -> protobuf:compile
gedrückt?



Bei m (rechts) ->
communicationSolution -> Plugins -
> protobuf -> protobuf:compile -
custom gedrückt?

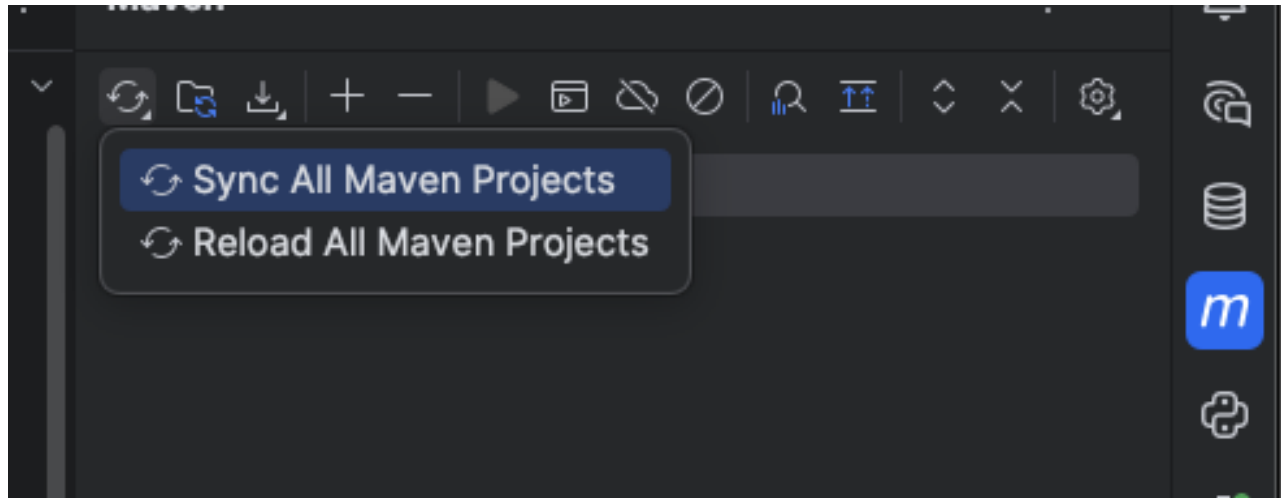
Und vergesse diese Zeilen nicht:

```
syntax = "proto3";  
package de.mcc;  
option java_package = "de.mcc";
```



Java_package = ____;
ist der Pfad zu deinen Klassen

Irgendetwas in pom.xml ist rot?



GitHUb –
Code kommt
später

